# PHP Interview Questions

## Basic PHP Interview Questions

1. What is PHP?
2. What are the differences between `echo`, `print`, and `print_r()`?

The print_r() function prints the information about a variable in a more human-readable way. Syntax: print_r(variable, return);

| Feature | `echo` | `print` | `print_r()` |
|---|---|---|---|
| Type | Construct | Construct | Function |
| Return value | No | Yes (always 1) | Yes (string if second param is true) |
| Multiple values | Yes | No | No |
| Array/Object | No | No | Yes (prints structure) |
| Usage | Output text | Output text | Debug arrays/objects |

3. What are the different data types in PHP?
4. What are variables and how are they declared in PHP?
5. How do you write comments in PHP?
6. What are constants? How do you define them?
7. What are the different types of arrays in PHP?
8. What is the difference between indexed, associative, and multidimensional arrays?

| Array Type | Key Type | Structure Example | Access Example |
|---|---|---|---|
| Indexed | Integer | `$arr = [10, 20, 30];` | `$arr[1]` → 20 |
| Associative | String | `$arr = ["a" => 10, "b" => 20];` | `$arr["b"]` → 20 |
| Multidimensional | Mixed (Nested) | `$arr = [["a"=>1], ["b"=>2]];` | `$arr[1]["b"]` → 2 |

9. What is the difference between `include`, `require`, `include_once`, and `require_once`?

| Statement | Error if file missing? | Halts execution? | Loads file only once? |
|---|---|---|---|
| `include` | Yes (Warning) | No | No |
| `require` | Yes (Fatal error) | Yes | No |
| `include_once` | Yes (Warning) | No | □ Yes |
| `require_once` | Yes (Fatal error) | Yes | □ Yes |

10. How does error handling work in PHP?

Error handling is the process of detecting, reporting, and responding to errors that occur during a program's execution

| Error Type | Description |
|---|---|
| **E_ERROR** | Fatal run-time errors that stop script execution. |
| **E_WARNING** | Run-time warnings (non-fatal, script continues). |
| **E_NOTICE** | Minor errors (e.g., using undefined variables). |
| **E_PARSE** | Compile-time parse errors (syntax errors). |
| **E_STRICT** | Suggestions for best practices. |
| **E_ALL** | All types of errors (except deprecated in old versions). |

11. What are the different types of errors in PHP?
12. What are sessions and cookies?
13. What is the difference between `isset()`, `empty()`, and `is_null()`?

| Function | Returns `true` if... | Checks existence? |
|---|---|---|
| `isset()` | Variable is **set** and **not null** | ✅ Yes |
| `empty()` | Variable is **not set** or **falsy** | ✅ Yes |
| `is_null()` | Variable **is exactly** `null` | ✅ Yes |

14. How do you connect to a MySQL database using PHP?
15. How do you fetch data from a MySQL database in PHP?

# Intermediate PHP Interview Questions

1. Explain the difference between GET and POST.
2. What is the difference between == and === in PHP?
3. What is the scope of a variable in PHP (local, global, static)?
4. What are PHP magic methods? Name a few.

   **E.g., `__construct()`, `__destruct()`, `__get()`, `__set()`, `__toString()`**

5. What is the use of the `final` keyword in PHP?
6. How does PHP support Object-Oriented Programming?
7. What is the difference between `public`, `private`, and `protected` in PHP?
8. How can you prevent SQL injection in PHP?

SQL Injection **is a major security vulnerability that occurs when user input is embedded directly into SQL queries without proper validation or escaping.**

9. What are traits in PHP?

**10.** How do you handle file uploads in PHP?

**File uploads in PHP are handled using the `$_FILES` superglobal. It stores the uploaded file information such as name, type, size, temporary name, and error code.**

**11.** What are namespaces in PHP?

Namespaces **in PHP are a way to** organize and group classes, interfaces, functions, and constants **to avoid** name conflicts **between different parts of a program — especially when using third-party libraries.**

**12.** What are `PDO` and `MySQLi`? What is the difference between them?
**13.** How can you create a RESTful API in PHP?

# Advanced PHP Interview Questions

1. What is Composer in PHP?
2. Explain dependency injection with an example.
3. What is the Singleton pattern?

**The Singleton pattern in PHP ensures that a class has only one instance and provides a global point of access to it**

- You want to **control access to shared resources** (e.g., database connection, logging, configuration).

- Prevent unnecessary **duplication of objects**.

4. What are design patterns in PHP? Can you name a few?

- Promote **code reusability**

- Improve **readability** and **scalability**

- Encourage **best practices**

- Make code **easier to test and maintain**

5. **Singleton, Factory, Observer, MVC, etc.**
6. How does autoloading work in PHP?
7. What is the difference between `abstract class` and `interface`?
8. How do you manage error logging in PHP?
9. How do you secure a PHP application?
10. How does Laravel differ from core PHP?
11. How do you implement pagination in PHP?

## Web-Related PHP Questions

1. How does PHP handle form submissions?
2. How do you manage user authentication and sessions in PHP?
3. What are CSRF and XSS? How do you prevent them in PHP?
4. What are HTTP response codes?
5. How do you send an email using PHP?

# MySQL Interview

## Basic MySQL Interview Questions

1. What is MySQL?
2. What are the different types of SQL statements?

**1. DDL (Data Definition Language)**

These statements define and modify the structure of database objects (tables, schemas, indexes, etc.).

- **Examples**:
  - `CREATE` – to create a new database or table.
  - `ALTER` – to modify an existing database object.
  - `DROP` – to delete database objects.
  - `TRUNCATE` – to remove all records from a table but keep its structure.
  - `RENAME` – to rename a database object.

**2. DML (Data Manipulation Language)**

These statements are used to manipulate data stored in tables.

- **Examples**:
  - `SELECT` – to retrieve data from a database.
  - `INSERT` – to add new records to a table.
  - `UPDATE` – to modify existing records.
  - `DELETE` – to remove records from a table.

**3. DCL (Data Control Language)**

These statements control access to data and database permissions.

- **Examples**:
  - `GRANT` – to give user access privileges.
  - `REVOKE` – to withdraw user access privileges.

4. **TCL (Transaction Control Language)**

These statements manage transactions in a database to ensure data integrity.

- **Examples**:
  - `COMMIT` – to save changes permanently.
  - `ROLLBACK` – to undo changes.
  - `SAVEPOINT` – to set a point within a transaction to roll back to later.

- `SET TRANSACTION` – to specify transaction properties.

5. **DQL (Data Query Language)**

- Technically, `SELECT` is considered part of DML, but sometimes it's categorized separately as DQL because it **only queries** data without modifying it.
- **Example**:
  - `SELECT * FROM employees;`

3. What is the difference between CHAR and VARCHAR in MySQL?

- **Use `CHAR`** when data is of **fixed length**.
- **Use `VARCHAR`** when data length **varies significantly**.

| | | |
|---|---|---|
| **Performance** | Slightly faster because it uses a fixed size and requires less processing. | Slightly slower because it stores variable length and may require extra processing. |
| **Trailing Spaces** | Trailing spaces are removed automatically when retrieving values. | Trailing spaces are preserved. |
| **When to Use** | Best for storing fixed-length data (e.g., country codes, phone numbers). | Best for storing variable-length data (e.g., names, addresses). |

4. What are primary keys and foreign keys?
5. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.
6. What is the difference between WHERE and HAVING clauses?

| Aspect | WHERE | HAVING |
|---|---|---|
| **Purpose** | Filters **rows** before grouping or aggregation. | Filters **groups** or aggregated data after `GROUP BY`. |
| **Used With** | Used with **SELECT**, **UPDATE**, **DELETE** statements. | Used only with **SELECT** (when using `GROUP BY` or aggregate functions). |
| **Aggregate Functions** | Cannot use aggregate functions directly (e.g., `SUM()`, `COUNT()`) unless inside a subquery. | Can use aggregate functions (e.g., `SUM()`, `AVG()`, `COUNT()`). |
| **Execution Order** | Applied **before** `GROUP BY`. | Applied **after** `GROUP BY`. |

### Example Without GROUP BY

```
SELECT * FROM employees
WHERE salary > 50000;
```

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

7. What are indexes? Why are they used?

Indexes are used to retrieve data from the database very fast.

8. What are the different storage engines in MySQL?

**InnoDB (Default Engine),** MyISAM, Memory (HEAP). CSV, Archive , NDB (Cluster Storage Engine)

9. How do you create a new database and a new table in MySQL?
10. What is the default port number for MySQL?

**3306**

**Example (Connecting with Custom Port)**

mysql -u root -p --port=3307

## Intermediate MySQL Interview Questions

1. What is the difference between **`TRUNCATE`**, **`DELETE`**, and **`DROP`**?
2. What is the difference between **`UNION`** and **`UNION ALL`**?

| Aspect | UNION | UNION ALL |
|---|---|---|
| **Duplicates** | Removes duplicate rows from the final result set. | Includes all rows, including duplicates. |
| **Performance** | Slower, because it performs an additional **duplicate elimination** step (uses `DISTINCT` internally). | Faster, because it does not check for duplicates. |
| **Usage** | Use when you need a **unique** combined result. | Use when duplicates are acceptable or required. |

3. Explain the concept of transactions and ACID properties.
4. How does MySQL handle auto-increment columns?

MySQL automatically increases the value by **1** whenever a new row is inserted.

You can set a custom start value:

```
ALTER TABLE employees AUTO_INCREMENT = 100;
```

A table can have **only one** `AUTO_INCREMENT` column.

5. What is the difference between **`InnoDB`** and **`MyISAM`** storage engines?

| Feature | InnoDB (Default) | MyISAM |
|---|---|---|
| **Transaction Support** | Supports **ACID transactions** with `COMMIT`, `ROLLBACK`, and `SAVEPOINT`. | Does **not** support transactions. |
| **Foreign Keys** | Supports **foreign key constraints**. | Does not support foreign keys. |
| **Locking Mechanism** | Uses **row-level locking**, allowing high concurrency. | Uses **table-level locking**, which can cause contention during writes. |

6. How would you optimize a slow-performing query?

- Analyze the Query Execution Plan
- Use Indexing Efficiently
- Optimize Joins
- Limit Returned Rows
- Partition Large Tables
- Optimize ORDER BY and GROUP BY
- Monitor Server Performance

7. What is a composite key?

A **composite key** is a **primary key** or **unique key** that consists of **two or more columns** combined to uniquely identify a row in a table.

- Individually, the columns in a composite key may not be unique.
- Together, their combination ensures **uniqueness**.

```
CREATE TABLE student_courses (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY(student_id, course_id)
);
```

- Here, `student_id` alone is not unique (a student can enroll in multiple courses).
- `course_id` alone is not unique (many students can take the same course).
- But the **combination** `(student_id, course_id)` uniquely identifies each record.

**8.** Explain the difference between **`CHAR_LENGTH`** and **`LENGTH`**.

| Function | Measures | Returns |
|---|---|---|
| **`CHAR_LENGTH(str)`** | Returns the **number of characters** in the string. | Character count |
| **`LENGTH(str)`** | Returns the **number of bytes** in the string | Byte count |

**9.** What are views in MySQL? How are they different from tables?

A **view** in MySQL is a **virtual table** based on the result of a `SELECT` query.

| Aspect | Views | Tables |
|---|---|---|
| **Storage** | Does **not** store data physically (only stores the query). | Stores data physically on disk. |
| **Data** | Data is **retrieved dynamically** from underlying tables. | Data is stored and managed directly. |
| **Update Capability** | Some views are **read-only**; updates depend on the view definition. | Tables can be freely updated. |
| **Performance** | May be slower for complex queries, since data is recalculated each time. | Generally faster since data is already stored. |

**10.** Explain the difference between **`NOW()`**, **`CURRENT_DATE()`**, and **`SYSDATE()`**.

**NOW():** Returns the current date and time (YYYY-MM-DD HH:MM:SS) when the query starts.
**`CURRENT_DATE()`:** Returns only the **current date** (`YYYY-MM-DD`) without time.
**`SYSDATE()`:** Returns the **current date and time** (`YYYY-MM-DD HH:MM:SS`) **at the moment the function is executed**, not when the query started.

## Advanced MySQL Interview Questions

1. What is the difference between clustered and non-clustered indexes in MySQL?
2. What are prepared statements, and why are they used?
3. How do you implement replication in MySQL?
4. What is the difference between **`Master-Slave`** and **`Master-Master`** replication?
5. How do you detect and resolve deadlocks in MySQL?
6. What is partitioning in MySQL, and what types are supported?
7. Explain the concept of foreign key constraints and cascading actions (**`ON DELETE CASCADE`**, **`ON UPDATE CASCADE`**).
8. What is the difference between **`EXPLAIN`** and **`DESCRIBE`**?
9. How does MySQL handle concurrency control?
10. What are common MySQL performance tuning techniques?

## Scenario-Based Questions

1. How would you find duplicate rows in a table?

(A) Finding duplicates based on all columns

```
SELECT *, COUNT(*) as duplicate_count
FROM your_table
GROUP BY column1, column2, column3, ... -- include all columns
HAVING COUNT(*) > 1;
```

(B) Finding duplicates based on a specific column (e.g., `email`)

```
SELECT email, COUNT(*) AS duplicate_count
FROM your_table
GROUP BY email
HAVING COUNT(*) > 1;
```

(C) Finding complete duplicate rows (and their IDs)

```
SELECT t.*
FROM your_table t
JOIN (
   SELECT column1, column2, COUNT(*) AS duplicate_count
```

```
 FROM your_table
   GROUP BY column1, column2
   HAVING COUNT(*) > 1
) dup
ON t.column1 = dup.column1
AND t.column2 = dup.column2;
```

2. How do you find the second highest salary from an employee table?
3. How would you back up and restore a MySQL database?

Backing up and restoring a MySQL database can be done using command-line tools (`mysqldump`, `mysql`) or via GUI tools (like phpMyAdmin). Here are the common methods

```
mysqldump -u root -p employees > employees_backup.sql
```

You can also back up a single table:

```
mysqldump -u username -p database_name table_name > table_backup.sql
```

Backup all databases

```
mysqldump -u username -p --all-databases > alldb_backup.sql
```

Restore a MySQL Database

```
mysql -u root -p employees < employees_backup.sql
```

Restore All Databases

```
mysql -u username -p < alldb_backup.sql
```

## 6. Using phpMyAdmin

1. Log into phpMyAdmin.
2. Select the database → Click **Export** → Choose **SQL** → Download file.
3. To restore, create/select a database → Click **Import** → Upload `.sql` file.

4. How would you handle a situation where the database is experiencing high load?
5. If a table has millions of records, how would you speed up SELECT queries?

(1) Use Indexing Properly, (2) Optimize Queries, (3) Use Proper Data Types,

(4) Partition Large Tables, (5)Use Caching, (6) Archive Old Data, (7)Use Read Replicas for Scaling